

SEGMENTASYON VE ENCODER-DECODER

1. GİRİŞ

Konunun Tanımı Görüntü işleme dünyasında segmentasyon, dijital bir görüntüyü anlamlı parçalara ayırma işlemidir. Geleneksel yöntemler görüntüyü bir bütün olarak sınıflandırırken (örneğin "bu resimde bir inek var"), semantik segmentasyon bu yaklaşımı piksel seviyesine indirger. Bu süreçte her bir piksel, ait olduğu nesne sınıfına göre (yol, yaya, bina vb.) tek tek etiketlenir.

Konunun Önemi Segmentasyon, bilgisayarlı görünün en kritik yapı taşlarından biridir çünkü nesnelerin sadece "ne" olduğunu değil, "nerede" olduklarını ve sınırlarının tam olarak nerede bittiğini piksel bazında söyler. Özellikle otonom sürüş sistemlerinde yol sınırlarının belirlenmesi veya tıbbi görüntülemede tümörlü hücrelerin piksel hassasiyetinde tespit edilmesi gibi hayati uygulama alanlarında vazgeçilmezdir.

2. TEMEL KAVRAMLAR

- **Semantic Segmentation** : Görüntüdeki her piksele bir sınıf etiketi atama işlemidir. Örneğin, bir sokak manzarasında tüm araç pikselleri aynı renge boyanırken, tüm yaya pikselleri farklı bir renge boyanır.
- **Convolution katmanı**: Görüntü üzerindeki kenar, köşe ve doku gibi özellikleri yakalamak için kullanılan filtreleme işlemidir.
- **Pooling katmanı**: Veriyi küçülterek işlem yükünü hafifleten bir yöntemdir. En yaygın olanı Max Pooling yöntemidir; bu işlemde genellikle 2 x2 bir çerçeve içindeki en büyük değer alınır ve veri boyutu %75 oranında azaltılır.
- **Feature Map (Özellik Haritası)**: Evrişim katmanlarından gelen filtrelenmiş verilerin oluşturduğu yeni haritadır.

3. YÖNTEM / MODEL AÇIKLAMASI

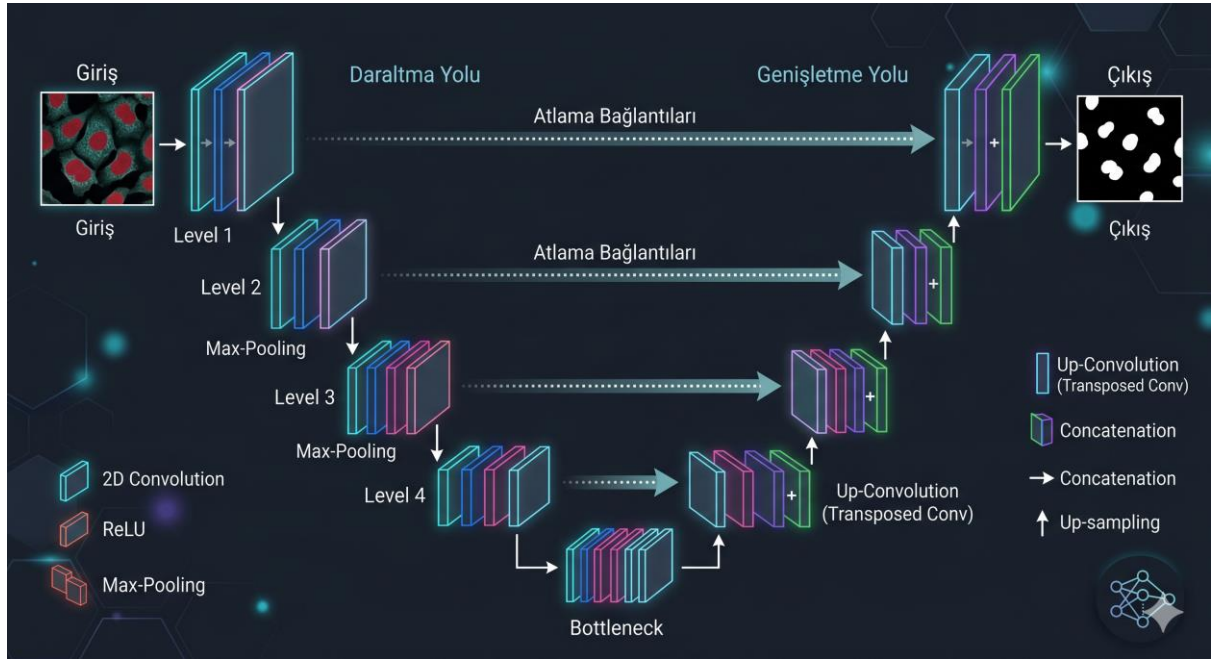
Encoder-Decoder Mimarisi Bu mimari, veriyi önce sıkıştıran (Encoder) sonra yeniden inşa eden (Decoder) bir yapıya sahiptir.

- **Encoder (Kodlayıcı)**: Görüntüyü girdi olarak alır, evrişim ve havuzlama katmanlarıyla çözünürlüğü düşürürken nesnenin ne olduğuna dair anlamsal bilgiyi artırır.

- **Decoder (Çözücü):** Kodlayıcıdan gelen düşük çözünürlüklü özeti alır ve Transpose Convolution gibi tekniklerle orijinal görüntü boyutuna geri döndürme işlemini yapar.

U-Net Modeli 2015 yılında geliştirilen U-Net, özellikle tıbbi görüntülerdeki veri azlığı sorununu çözmek için tasarlanmıştır. Modelin en büyük yeniliği **"Skip Connections" (Atlama Bağlantıları)** fikridir.

- **Skip Connections:** Kodlayıcı tarafındaki yüksek çözünürlüklü detaylar, doğrudan çözücü tarafındaki karşılık gelen katmanlara aktarılır. Bu sayede, encoder görüntüyü küçültürken kaybolan kenar ve doku detaylarını decoderde bu bağlantılar sayesinde geri kazanılır ve sonuçların kaba çıkmasını engeller.



4. UYGULAMA ALANLARI

- **Tıbbi Görüntü Analizi:** Kanserli hücrelerin veya organ sınırlarının hassas tespiti.
- **Otonom Araçlar:** Yol, yaya ve trafik işaretlerinin gerçek zamanlı segmentasyonu ile güvenli sürüş kararları alınması.
- **Uydu Görüntüleme:** Haritacılık faaliyetlerinde bina ve arazilerin belirlenmesi.

5. YAPILAN MİNİ PROJE AÇIKLAMASI

Çalışmanın Tanımı: Mask Görselleştirme Bu çalışmada, bir görüntünün semantik segmentasyon sürecindeki maskeleme aşaması ele alınmıştır. Amaç, görüntüyü alıp, pikselleri sınıflarına göre kümeleyerek sınırları belirgin bir maske çıktısı elde etmektir.

- **Yöntem:** Girdi görüntüsü piksel tabanlı sınıflandırmaya tabi tutulmuş ve ardından her sınıf farklı bir renk koduyla temsil edilmiştir.
- **Kullanılan Veri Seti:** nesne odaklı (araba) görsel kullanılarak modelin nesne ayırt etme kapasitesi test edilmiştir.
- **Sonuçlar:** * araba kırmızı arka plan yeşil olarak maskelenmiştir.
 - U-Net mimarisi sayesinde, özellikle nesne sınırlarının net bir şekilde çizildiği ve maskenin orijinal görüntüyle yüksek korelasyon gösterdiği gözlemlenmiştir.

Kodlar

```
!pip install pycocotools segmentation-models-pytorch
```

... [Gizli çıkışı göster](#)

```
import torch
import torchvision
from torchvision import transforms as T
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. COCO veri setiyle önceden eğitilmiş gerçek bir model çağır (DeepLabV3)
# Bu model 20'den fazla nesneyi (araba, köpek vb.) tanır.
model = torchvision.models.segmentation.deeplabv3_resnet101(pretrained=True)
model.eval() # Modeli tahmin moduna aldık

# 2. Resmini modele hazırladık (Boyutlandırma ve Normalizasyon)
yol = '/content/araba.jpg'
resim = cv2.imread(yol)
resim = cv2.cvtColor(resim, cv2.COLOR_BGR2RGB)
h, w, c = resim.shape # Orijinal boyutları kaydet

transform = T.Compose([
    T.ToPILImage(),
    T.Resize(520), # Modelin beklediği boyut
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
input_tensor = transform(resim).unsqueeze(0) # Modele uygun formata soktuk

# 3. YAPAY ZEKA TAHMİNİ: Model nesneleri buluyor ve pikselleri sınıflandırıyor
with torch.no_grad():
    output = model(input_tensor)['out'][0]
output_predictions = output.argmax(0).byte().cpu().numpy()
```

```

# 4. Maskeyi renklendirme (Görselleştirme)
# COCO sınıflarına göre renk haritası oluşturma
num_classes = output.shape[0]
r = np.zeros_like(output_predictions).astype(np.uint8)
g = np.zeros_like(output_predictions).astype(np.uint8)
b = np.zeros_like(output_predictions).astype(np.uint8)

for l in range(0, num_classes):
    idx = output_predictions == l
    # Rastgele renkler atıyoruz (Hangi sınıfın hangi ID olduğu)
    r[idx] = np.random.randint(0, 255)
    g[idx] = np.random.randint(0, 255)
    b[idx] = np.random.randint(0, 255)

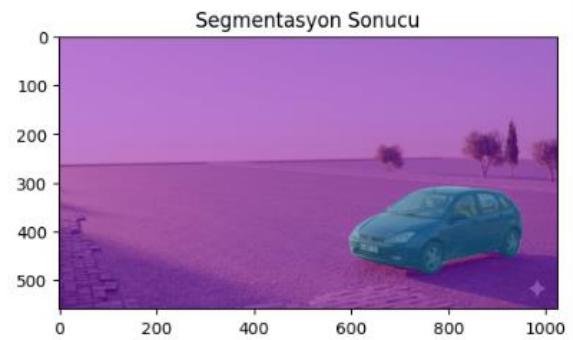
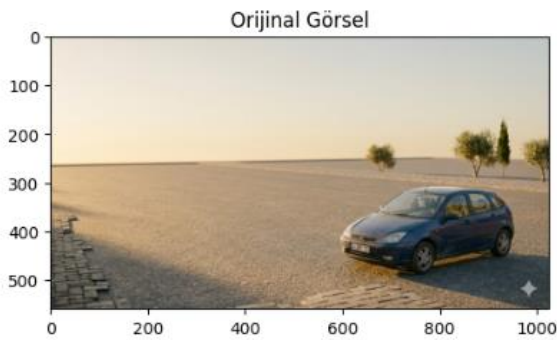
# Oluşturulan renkli maskeyi orijinal resim boyutuna büyütüyoruz
renkli_maske = cv2.merge([r, g, b])
renkli_maske = cv2.resize(renkli_maske, (w, h), interpolation=cv2.INTER_NEAREST)

# 5. Orijinal resim ile maskeyi harmanladık
sonuc = cv2.addWeighted(resim, 0.5, renkli_maske, 0.5, 0)

# 6. Ekrana bastırdık
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Orijinal Görsel")
plt.imshow(resim)

plt.subplot(1, 2, 2)
plt.title("Segmentasyon Sonucu")
plt.imshow(sonuc)
plt.show()

```



Neden TensorFlow yerine PyTorch

Orijinal U-Net makalesindeki ve güncel çalışmaların büyük çoğunluğunun PyTorch ile gerçekleştirildiğini, bu nedenle en güncel ve doğrulanmış modelleri kullanmak adına bu yolu seçtim

Ayrıca projenin başlangıcında TensorFlow Hub üzerinden DeepLabV3+ modellerini entegre etmeye çalıştım. Ancak kütüphanenin güncel sürümlerindeki uzak sunucu bağlantı hataları ve model ağırlıklarının indirilmesi sırasında oluşan 'invalid header' hataları nedeniyle modelin eğitilmiş verilerine erişim sağlanamadığından.

6. SONUÇ

Bu çalışma kapsamında segmentasyonun sadece bir sınıflandırma değil, görüntüdeki her pikselli anlama süreci olduğu görülmüştür. Encoder-Decoder mimarilerinin veriyi sıkıştırma yeteneği ile U-Net'in detay koruma başarısı birleştiğinde, az veriyle bile yüksek hassasiyetli maskeler üretilabildiği gözlemlenmiştir. Bu teknolojilerin gelecekte otonom sistemler ve sağlık teşhislerinde standart haline geleceği değerlendirilmektedir.

7. KAYNAKÇA

- [1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, 2015, pp. 234–241.
- [2] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.